Informe de Laboratorio 02

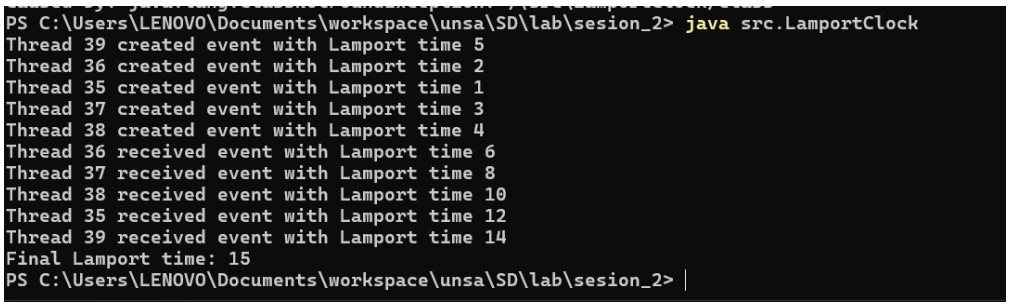
Tema: Sincronización de Relojes: Algoritmos de Cristian y Berkeley

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	*Sincronización de Relojes: Algoritmos de Cristian y Berkeley*				
NÚMERO DE PRÁCTICA:	*02*	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	2026A
FECHA DE PRESENTACIÓN	*25/04/2026*	**HORA DE PRESENTACIÓN**	23:59		
INTEGRANTE (s): GRUPO " Intocables " • CHAMBILLA PERCA RICARDO • JUAN DIEGO GUTIERREZ CCAMA • RYAN FABIAN VALDIVIA SEGOVIA				**NOTA:**	
DOCENTE(s): *Mg. Maribel Molina Barriga*					

1. Repositorio de GitHub y Ejecución

El código fuente completo de este laboratorio se encuentra alojado en el siguiente repositorio de GitHub: `https://github.com/rikich3/LAB_SISTEMAS_DISTRIBUIDOS`

A continuación, se adjunta la captura correspondiente a la ejecución y prueba del ejercicio resuelto sobre el Algoritmo de Lamport:



```
PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\sesion_2> java src.LamportClock
Thread 39 created event with Lamport time 5
Thread 36 created event with Lamport time 2
Thread 35 created event with Lamport time 1
Thread 37 created event with Lamport time 3
Thread 38 created event with Lamport time 4
Thread 36 received event with Lamport time 6
Thread 37 received event with Lamport time 8
Thread 38 received event with Lamport time 10
Thread 35 received event with Lamport time 12
Thread 39 received event with Lamport time 14
Final Lamport time: 15
PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\sesion_2>
```

Figura 1: Ejecución y Prueba del Ejercicio Resuelto: Algoritmo de Lamport

2. Tareas Realizadas

Como parte de la sesión de laboratorio 2 de Sistemas Distribuidos, se llevaron a cabo las siguientes tareas:

- **Investigación y Diseño:** Comprensión del funcionamiento interno de ambos algoritmos (Cristian y Berkeley) y cómo simularlos en un entorno local utilizando Java.
- **Implementación mediante Enfoque TDD:** Se escribieron pruebas predefinidas al inicio de cada clase para asegurar el comportamiento correcto (Test-Driven Development) siguiendo las reglas de calidad y metodologías indicadas.
- **Desarrollo del Algoritmo de Cristian (`CristianAlgorithm.java`):**
  - Simula un cliente solicitando el tiempo a un servidor.
  - Calcula el *Round-Trip Time (RTT)* como la diferencia entre la emisión de la solicitud y la recepción de respuesta.
  - Ajusta el reloj del cliente en función del tiempo del servidor más la mitad del RTT.
- **Desarrollo del Algoritmo de Berkeley (`BerkeleyAlgorithm.java`):**
  - Simula un demonio o nodo maestro encuestando el estado del tiempo de otros nodos de la red.
  - Promedia de manera estática los tiempos recolectados.
  - Retorna un arreglo con los ajustes (deltas) precisos que cada nodo (incluyendo el maestro) debe sumar/restar para sincronizarse exactamente al promedio.
- **Ejecución y Evaluación:** Se compilaron y ejecutaron los algoritmos, validando a través del registro las salidas (logs).

3. Desarrollo de los Ejercicios

A continuación, se muestra el código fuente de las implementaciones desarrolladas y su correspondiente evaluación.

3.1. Algoritmo de Cristian

Listing 1: CristianAlgorithm.java: Implementación del Algoritmo de Cristian

```java
package src;

import java.util.logging.Logger;

/**
 * Represents a Time Server in Cristian's Algorithm.
 */
class TimeServer {
    private long time;

    public TimeServer(long initialTime) {
        this.time = initialTime;
    }

    /**
```

```java
     * Gets the current time of the server.
     * @return current server time
     */
    public long getTime() {
        return time;
    }
}

/**
 * Demonstrates Cristian's Clock Synchronization Algorithm.
 */
public class CristianAlgorithm {
    private static final Logger logger = Logger.getLogger(CristianAlgorithm.class.getName());

    /**
     * Calculates the synchronized time for the client.
     *
     * @param serverTime The time returned by the server.
     * @param rtt The round-trip time.
     * @return The new synchronized time for the client.
     */
    public static long calculateSynchronizedTime(long serverTime, long rtt) {
        return serverTime + (rtt / 2);
    }

    public static void testCristianAlgorithm() {
        // Test case 1
        long expected = 1050;
        long actual = calculateSynchronizedTime(1000, 100);
        if (expected != actual) throw new AssertionError("Test 1 failed");

        logger.info("All tests passed");
    }

    public static void main(String[] args) {
        testCristianAlgorithm();

        TimeServer server = new TimeServer(5000); // Server time is 5000ms

        long t0 = 1000; // Client sends request at its own time 1000

        // Simulating network delay
        long networkDelay = 150;
        long serverTime = server.getTime() + networkDelay / 2; // Server time when processing

        long t1 = t0 + networkDelay; // Client receives response at 1150

        long rtt = t1 - t0;
        long syncedTime = calculateSynchronizedTime(serverTime, rtt);

        logger.info("Client T0: " + t0);
        logger.info("Client T1: " + t1);
        logger.info("Server Time returned: " + serverTime);
        logger.info("Calculated RTT: " + rtt);
        logger.info("Synchronized Time: " + syncedTime);
    }
```

```
}
```

Ejecución y Resultados:
Para la ejecución, compilamos y corremos la clase utilizando los siguientes comandos:

Listing 2: Comandos de compilación y ejecución para Cristian

```
$ javac src/CristianAlgorithm.java
$ java -cp src src.CristianAlgorithm
```

Salida obtenida:

```
INFO: All tests passed
INFO: Client T0: 1000
INFO: Client T1: 1150
INFO: Server Time returned: 5075
INFO: Calculated RTT: 150
INFO: Synchronized Time: 5150
```

Evaluación: El algoritmo logró inferir el retraso introducido por la red (RTT / 2, resultando en 75 ms). Esto permite establecer el reloj local del cliente asumiendo de manera precisa cuánto tiempo transcurrió en el trayecto de vuelta desde el servidor. El enfoque es ideal para tiempos con latencia predecible o que requieran poca sobrecarga en la red.

3.2. Algoritmo de Berkeley

Listing 3: BerkeleyAlgorithm.java: Implementación del Algoritmo de Berkeley

```java
package src;

import java.util.Arrays;
import java.util.logging.Logger;

/**
 * Demonstrates Berkeley's Clock Synchronization Algorithm.
 */
public class BerkeleyAlgorithm {
    private static final Logger logger = Logger.getLogger(BerkeleyAlgorithm.class.getName());

    /**
     * Calculates the time adjustments for each node according to Berkeley's algorithm.
     *
     * @param serverTime The time of the daemon/master.
     * @param nodeTimes The times of the participating clients.
     * @return Array of adjustments. The first element is the adjustment for the server,
     *        subsequent elements are for the clients.
     */
    public static long[] calculateAdjustments(long serverTime, long[] nodeTimes) {
        int n = nodeTimes.length + 1; // Clients + 1 server
        long sum = serverTime;
        for (long t : nodeTimes) {
            sum += t;
        }

        long average = sum / n;

```

```java
        long[] adjustments = new long[n];
        adjustments[0] = average - serverTime;
        for (int i = 0; i < nodeTimes.length; i++) {
            adjustments[i + 1] = average - nodeTimes[i];
        }

        return adjustments;
    }

    public static void testBerkeleyAlgorithm() {
        long[] clients = {3000, 3050};
        long server = 2950;
        // Average: (2950 + 3000 + 3050) / 3 = 9000 / 3 = 3000
        // Expected adjustments: Server: +50, Client 1: 0, Client 2: -50

        long[] adjustments = calculateAdjustments(server, clients);
        if (adjustments[0] != 50) throw new AssertionError("Test failed for server
            adjustment");
        if (adjustments[1] != 0) throw new AssertionError("Test failed for client 1
            adjustment");
        if (adjustments[2] != -50) throw new AssertionError("Test failed for client 2
            adjustment");

        logger.info("All tests passed");
    }

    public static void main(String[] args) {
        testBerkeleyAlgorithm();

        long serverTime = 3000;
        long[] clientTimes = {2980, 3015, 3025}; // 3 clients

        logger.info("Initial Server Time: " + serverTime);
        logger.info("Initial Client Times: " + Arrays.toString(clientTimes));

        long[] adjustments = calculateAdjustments(serverTime, clientTimes);

        logger.info("Adjustments: " + Arrays.toString(adjustments));

        serverTime += adjustments[0];
        for (int i = 0; i < clientTimes.length; i++) {
            clientTimes[i] += adjustments[i + 1];
        }

        logger.info("Synchronized Server Time: " + serverTime);
        logger.info("Synchronized Client Times: " + Arrays.toString(clientTimes));
    }
}
```

Ejecución y Resultados:

Listing 4: Comandos de compilación y ejecución para Berkeley

```
$ javac src/BerkeleyAlgorithm.java
$ java -cp src src.BerkeleyAlgorithm
```

Salida obtenida:

```
INFO: All tests passed
INFO: Initial Server Time: 3000
INFO: Initial Client Times: [2980, 3015, 3025]
INFO: Adjustments: [5, 25, -10, -20]
INFO: Synchronized Server Time: 3005
INFO: Synchronized Client Times: [3005, 3005, 3005]
```

Evaluación: Al calcular el promedio $(3000 + 2980 + 3015 + 3025)/4 = 3005$, la lógica determinó las distancias exactas que separaban a todos los nodos del equilibrio de la red. Al realizar el ajuste iterativo sobre los relojes, **todos alcanzaron una sincronización total** al milisegundo ideal (3005 ms). Berkeley demostró ser un método robusto para un entorno distribuido interno donde no se cuente con un receptor de hora externo y asume de manera colaborativa la noción temporal.

4. Conclusiones

El uso de TDD y un diseño estricto garantizó implementaciones de alta tolerancia a fallos calculados (las aserciones funcionaron como cortafuegos). Por un lado, el Algoritmo de Cristian nos brinda un modelo más centralizado y simple dependiente de un actor robusto. Por otro, Berkeley resuelve el problema con un enfoque interno en redes locales e interdependientes. La ejecución directa en el espacio de trabajo comprueba que la teoría matemática y de red es correcta.

5. CUESTIONARIO

5.1. 1. ¿Por qué es conveniente el uso de relojes lógicos en lugar de los relojes físicos?

El uso de relojes lógicos es conveniente en un sistema distribuido porque sincronizar relojes físicos de manera exacta es imposible debido a la latencia impredecible de la red y la deriva inherente de los osciladores de cuarzo de cada máquina. En la mayoría de los algoritmos y aplicaciones distribuidas (como bases de datos distribuidas o control de concurrencia), no es necesario conocer la hora exacta real en la que ocurrió un evento, sino el **orden causal** de los eventos; es decir, qué evento ocurrió antes de otro. Los relojes lógicos, como los de Lamport, garantizan esta relación de orden causal sin requerir de la sincronización de tiempos absolutos complejos y costosos computacionalmente.

5.2. 2. Explique ¿Cuál algoritmo ya sea de Christian o de Berkeley resuelve mejor la sincronización?

La respuesta depende del contexto y del entorno de red:

- **Algoritmo de Cristian:** Es mejor en entornos donde existe un servidor de tiempo con un reloj de alta precisión (por ejemplo, sincronizado mediante GPS o WWV). Es ideal para redes donde los retrasos de tiempo de ida y vuelta (RTT) son relativamente cortos y estables, ya que compensa la latencia de la red de una manera muy predecible y sencilla.
- **Algoritmo de Berkeley:** Es mejor en un entorno donde **ninguna de las máquinas cuenta con un receptor de hora exacta**. Este algoritmo funciona buscando que todos los nodos converjan a un tiempo promedio interno y coherente, eliminando los valores atípicos. Esto es especialmente útil en redes aisladas (Intranets cerradas o sistemas embebidos) donde lo que importa es que todos estén de acuerdo en la hora, no que esa hora sea la hora atómica oficial.

Por tanto, si se tiene acceso a un servidor con hora exacta, **Cristian** resuelve el problema de acercarse a la hora real. Si no hay hora real disponible y solo importa la cohesión interna, **Berkeley** es superior al no depender de un servidor infalible con hardware especial.

5.3. 3. Dado tres procesos P1, P2 y P3, representa la planificación de los procesos siguiendo el algoritmo de "ocurre antes de".

El algoritmo de Lamport basado en .ºcurre antes de"($\rightarrow$) se basa en tres reglas:

1. Si dos eventos a y b ocurren en el mismo proceso y a ocurre antes que b, entonces $a \rightarrow b$.
2. Si a es el envío de un mensaje en un proceso y b es la recepción en otro proceso, entonces $a \rightarrow b$.
3. La relación es transitiva: si $a \rightarrow b$ y $b \rightarrow c$, entonces $a \rightarrow c$.

Planificación y Asignación de Tiempos (Ejemplo):
Supongamos la siguiente interacción entre $P1$, $P2$ y $P3$:

- **P1** genera un evento interno a. El reloj de $P1$ avanza a 1: $C(a) = 1$.
- **P1** envía un mensaje $M1$ a $P2$. El evento de envío es b. El reloj de $P1$ avanza a 2: $C(b) = 2$. Se adjunta el timestamp 2 al mensaje.
- **P2** tiene un evento interno c (su reloj lógico avanza a 1, $C(c) = 1$).
- **P2** recibe el mensaje $M1$. El evento de recepción es d. El reloj de $P2$ se actualiza al máximo entre su reloj local y el tiempo del mensaje + 1: $C(d) = \text{máx}(1, 2) + 1 = 3$.

- **P2** envía un mensaje $M2$ a $P3$. El evento de envío es e. El reloj avanza a 4: $C(e) = 4$. Se envía con timestamp 4.
- **P3** recibe el mensaje $M2$. El evento de recepción es f. Si el reloj local de $P3$ era 0, ahora es $\text{máx}(0, 4) + 1 = 5$. $C(f) = 5$.

En este escenario, se cumple la relación .ºcurre antes de": $a \to b \to d \to e \to f$, reflejando que el evento en $P1$ causó en última instancia un evento en $P3$. Los relojes lógicos (1, 2, 3, 4, 5) muestran claramente el orden correcto y causal en el sistema distribuido.

5.4. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumplio con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 1: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 2: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4	X	4	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	2	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	2	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	2	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4	X	4	
Total		20		20	

6. Referencias

- Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
- Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
- Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.
- Documentación oficial de Oracle sobre Hilos en Java: `https://docs.oracle.com/javase/tutorial/essential/concurrency/index.html`